

Algorithmic Approaches to Efficient Nearest Neighbor Search in High-Dimensional Data

Tristan Jones
Biomedical Informatics
University of Nebraska Omaha
Omaha, Nebraska, USA
Tristanjones@nebraska.edu

ABSTRACT

Nearest neighbor search is a fundamental computational problem that appears in numerous machine learning and data analysis applications, including classification, clustering, and similarity-based retrieval systems. As datasets grow larger and occupy higher-dimensional spaces, the computational cost of finding an exact nearest neighbor becomes increasingly prohibitive. This paper presents a comparative study between five different k-nearest neighbor search algorithms: the brute-force linear search, k-dimensional tree (k-d tree) search, and three configurations of a randomized k-d tree forest using a novel depth-first search query module with a configurable backtracking hyperparameter designed as an approximate nearest neighbor (ANN) methodology. Each algorithm was implemented in Python and evaluated on synthetic clustered datasets of 100,000 points, spanning dimensionalities from 2 to 200. The performance of said algorithms was measured by average query runtime, build and query memory consumption, and search accuracy relative to a brute-force ground-truth approach. The results demonstrate that the k-d tree is more effective than brute force for low-dimensionality datasets, whereas they start to converge around $d=20$, and it actually marginally exceeds brute force search at $d=200$. This is consistent with the curse of dimensionality that comes with these approaches. All three forest ANN variants were consistently faster than both exact methods across every tested dimensionality, with $bt=0$ completing queries up to 210x faster than brute force at $d=200$, but at the cost of only 1.2% accuracy at that dimensionality. The backtracking budget emerges as a meaningful hyperparameter controlling this tradeoff: $bt=2$ recovered accuracy to 39.2% at $d=200$ while remaining roughly 6x faster than brute force. These findings highlight the practical importance of backtracking budget as a tunable parameter in approximate nearest neighbor search and motivate further investigation into methods designed specifically for high-dimensional, large-scale datasets.

CCS CONCEPTS

• Machine learning algorithms • Design and analysis of algorithms

KEYWORDS

Nearest Neighbor Search, k-Nearest Neighbor, k-d Trees, Approximate Nearest Neighbor, High-Dimensional Data, Algorithm Analysis

1 INTRODUCTION

The k-nearest neighbor (k-NN) problem is relatively simple; it asks, given the dataset of points of a certain dimensionality, give me the nearest k points to a specific query. Despite this conceptual simplicity, the nearest neighbor problem actually sits at the core of a multitude of different applications. In machine learning, k-NN can be used for both classification and regression, which rely directly on nearest neighbor search. In computer vision, image retrieval systems must identify the most visually similar images from a database of millions of images. In bioinformatics, nearest neighbor methods can be used for a range of different applications, such as gene expression profiles, protein structures, or in medical imaging applications. The practical importance of having an efficient nearest neighbor search is imperative for a multitude of applications.

Given this breadth of application, the computational efficiency of nearest neighbor search becomes critical. Nearest neighbor search requires identifying the most similar data points in a dataset according to a defined distance metric, and while the concept is simple, its computational cost grows significantly as the dataset size and dimensionality increase. A naive implementation, also known as the brute force approach, requires comparing each query point to every point in the dataset, leading to high computational complexity and slow query times for large datasets. Effectively, we end up with a time complexity of $O(nd + n \log n)$ per query, where in this case n is the number of data points in the set and d is the dimensionality of each data point. If you needed to search a dataset with one million entries, where each entry had 100's of dimensions, you would need to compute over 100 million multiplications and additions before even accounting for any sorting methods which in this case adds the additional $O(n$

$\log n$). While this may work for some applications, for any real-time or interactive applications, this method would be entirely impractical, particularly at the scale of modern machine learning workloads [7].

Therefore, it is easy to see that for these high-dimensional datasets, the naive approach simply will not scale. However, researchers have developed a variety of data structures and algorithms to address this scalability problem. Spatial indexing structures such as the k-d tree reduce query complexity to $O(\log n)$ in low-dimensional settings by recursively partitioning space and pruning branches of the search tree that cannot contain a nearest neighbor [1]. However, although this method is exact, it suffers from the well-documented “curse of dimensionality,” meaning that as dimensionality increases, the tree’s pruning conditions become less effective. In the worst case, the search degenerates to a full linear scan. This, of course, motivates the development of an approximate nearest neighbor search algorithm, where we sacrifice a small amount of accuracy in exchange for dramatically improved scalability.

This paper aims to implement and experimentally evaluate five k-NN algorithms that represent different points on the accuracy-efficiency trade-off spectrum. The first is the brute-force approach, which essentially serves as a ground truth and a baseline for what the most accurate results should return. The second is the k-d tree, which represents the canonical exact spatial indexing approach as highlighted in Bentley et al. [1]. The final algorithm, in part inspired by Lu and Gweon [4], essentially uses a randomized k-d tree forest for approximate nearest neighbor search with a DFS query module with a configurable backtracking budget tested at $bt=0$, $bt=1$, and $bt=2$. All five algorithms will be evaluated on a synthetic clustered dataset across seven different dimensionalities (2, 5, 10, 20, 50, 100, 200), measuring runtime, memory usage, and accuracy of each algorithm. The goal is to empirically characterize where each algorithm excels and where it breaks down, providing practical insight into algorithm selection for nearest neighbor search in high-dimensional spaces.

2 BACKGROUND: NEAREST NEIGHBOR SEARCH

2.1 Problem Formulation

The k-nearest neighbor problem can be formally defined as follows. Let $D = \{x_1, x_2, \dots, x_n\}$ be a dataset of n points in $\mathbb{R}^d$, and let $q \in \mathbb{R}^d$ be a query point. The k-nearest neighbor search problem asks for the k points in D that minimize the distance to q under a chosen distance metric. Formally, the goal is to find a subset $S \subseteq D$ with $|S| = k$ such that for all $p \in S$ and all $r \in D \setminus S$, $d(p, q) \leq d(r, q)$.

In this work, Euclidean distance is used as the distance metric throughout. Euclidean distance between two points a and b in $\mathbb{R}^d$ is defined as:

$$d(a, b) = \sqrt{\sum_i (a_i - b_i)^2}$$

There are multiple distance metrics that can be used in this instance; in addition to the Euclidean distance, others include the Hamming distance, Minkowski distance, and Manhattan distance. The choice of distance metric is really just dependent on the problem that you are trying to solve. In this instance, the Euclidean distance is the ideal metric, since we are simply trying to measure how far points are from one another as the crow flies, so to speak. The choice of calculation is non-trivial overall. As we approach significantly higher dimensions, the Manhattan distance may become more useful, since the Euclidean distance becomes less stable in high-dimensional regimes[8]. However, for the dimensionalities specifically tested within this study, for up to 200 dimensions, the Euclidean distance remains a reasonable and standard choice; it is the metric most commonly used in the nearest neighbor literature.

The value $k=10$ was chosen as the search parameter for this study. The choice of k is not trivial in this instance; smaller values of k , such as $k=1$ or $k=3$, place higher demands on the search algorithm since there is less tolerance for returning a slightly suboptimal neighbor. The value of k is a hyperparameter that is generally adjusted by the user for the use case of the study itself. Larger values of k may be more forgiving for approximation methods since the probability that all of the correct k nearest neighbors exist within the candidate pool is higher. $k=10$ represents a relatively moderate value here that is commonly used. Essentially, by searching for these many neighbors, we can effectively look at the accuracy of these algorithms meaningfully without the k being so large that the approximate methods are effectively trivial. Future work on this may include tuning k as a hyperparameter to see how effective it is and what the trade-offs may be.

2.2 The Curse of Dimensionality

A central concept in the realm of the nearest neighbor search is the curse of dimensionality, a term describing the way that geometric intuitions really end up breaking down within high-dimensional spaces. In low dimensions, spatial indexing structures work very well because nearby points in a dataset tend to occupy nearby regions of space, and large portions of the search space can be ruled out quickly. However, as the dimensionality grows, there are several problems that end up emerging simultaneously.

Firstly, the volume of space grows exponentially with dimension, so a dataset with 100,000 individual points that is relatively dense in 2 dimensions becomes extremely sparse in 200 dimensions. Second, the effectiveness of pruning a k-d tree search depends on the distance from the query point to a splitting hyperplane being greater than the

current best distance to a nearest neighbor candidate. Therefore, in high dimensions, the splitting hyperplanes become individually less informative, and the probability that the far side of any given split can be safely pruned decreases rapidly. In Sample et al. [2], they analyze this degradation in detail, showing that even optimized k-d tree search strategies encounter more nodes at higher dimensionalities exponentially. Lastly, as noted above, distances between points become increasingly similar in high dimensions, making the problem of distinguishing the true nearest neighbor from other candidates fundamentally harder, regardless of the search strategy. These properties of high-dimensional geometry motivate approximate methods, which attempt to return a slightly suboptimal nearest neighbor in exchange for search times that scale more gracefully [2].

3 EXACT NEAREST NEIGHBOR METHODS

3.1 Brute-Force Linear Search

The simplest approach to the k-NN search is an exhaustive comparison: compute the distance from the query point to every singular point in the dataset, sort the resulting distances, and then from that return the k smallest distances. This is sometimes called linear search or the naive method of k-NN.

3.1.1 Algorithm.

For a dataset D of n points and a query q , brute-force k-NN proceeds as follows:

```
function brute_force_knn (D, q, k):
    distances = []
    for each point p in D:
        d = euclidean_distance (q, p)
        distances.append ((d, p))
    sort distances ascending by d
    return first k points
```

3.1.2 Complexity.

The time complexity of brute-force k-NN is $O(nd + n \log n)$ per query, where n is the size of the dataset, i.e., the number of individual points, and d is the dimensionality of the dataset, or the number of variables in each point. Here, the nd term accounts for computing the n Euclidean distances, each of which requires d unique operations for each dimension. The memory complexity is $O(n)$ because each distance must be stored before sorting. The sorting cost of $O(n \log n)$ is the dominant term for large n , making it the primary bottleneck rather than the distance computation itself.

3.1.3 Implementation.

In this study, the brute-force implementation computes all pairwise distances and uses Python's built-in sort, which implements Timsort [11]. This is effectively a hybrid of merge sort and insertion sort, with $O(n \log n)$ worst-case time complexity, additionally yielding strong adaptive performance on partially sorted data. This method is by definition always correct since it looks through every single possible option and therefore serves as the ground truth against which the other algorithms' accuracy will be measured.

3.1.4 Strengths and Limitations.

Brute force has two meaningful advantages: it is trivially simple to implement, and it guarantees that you will always get the exact results for any distance metric and any dataset. However, it has a critical weakness, which is that it has a linear scaling with dataset size, which makes it pretty impractical for a very large n . Notably, its runtime scales well with dimensionality; adding more dimensions slows it down proportionally, since each new dimension incurs more computational work linearly. However, there is no catastrophic degradation of the kind seen in tree-based methods.

3.2 K-Dimensional Tree Search

The k-d tree, introduced by Bentley [1] in 1975, is one of the most widely studied and applied data structures for exact nearest-neighbor search. It organizes points into a multidimensional dataset through the recursive binary partitioning of the space, thereby creating a binary tree structure. This, in turn, enables search algorithms to prune large regions of the dataset that cannot contain a nearest neighbor. Subsequent work by Sample et al. [2] analyzed the conditions under which k-d tree search degrades and proposed strategies for mitigating this effect.

3.2.1 Algorithm.

This algorithm is structured into multiple parts: the first is the build phase, where we create the k-d tree, and the second is the query phase, where we search the k-d tree. A k-d tree is constructed as stated earlier, recursively. At each step, the algorithm selects the splitting dimension by identifying the dimension with the greatest spread among the current set of points; for a 2D dataset, this might be the x-axis at one level and the y-axis at another, depending on where the data is most spread out. We then partition the points at the median value along that dimension. The median point becomes the current node, points with values below the median along with the split dimension, form the left part of the subtree, whilst the points at or above the median form the right subtree. This process continues recursively until all points have been placed onto the tree. The depth of the

resulting tree is $O(\log n)$, and each of the n points occupies exactly one node on the binary tree. As such, the build phase pseudocode looks something like this:

```
function build_kdtree(points, depth=0):
    if points is empty: return null
    axis = dimension with largest spread among points
    sort points by axis value
    median = points[len(points) // 2]
    node.point = median
    node.axis = axis
    node.left = build_kdtree(points[:median_idx])
    node.right = build_kdtree(points[median_idx+1:])
    return node
```

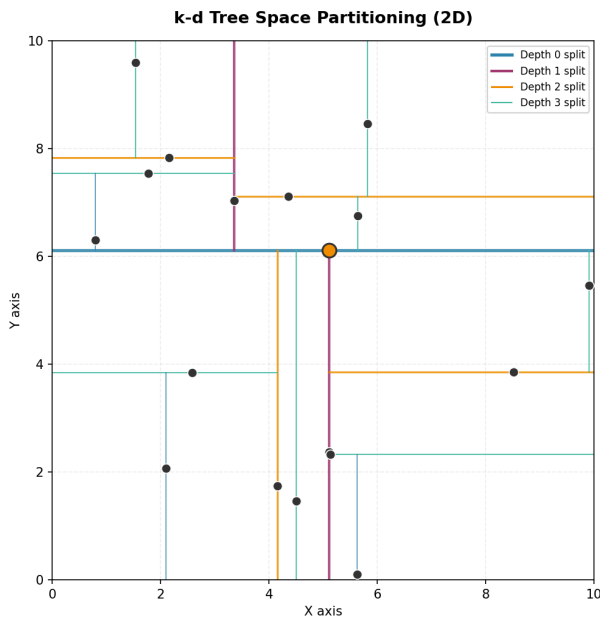


Figure 1: Visualization of k-d tree space partitioning on a sample 2D dataset. Each colored line represents a splitting hyperplane at successive tree depths, showing how the space is recursively divided into smaller regions. The highlighted point in orange represents the root node, which is the first split applied to the full dataset.

The choice to split on the dimension of greatest spread, rather than cycling through dimensions in fixed order, is a deliberate optimization here. In this case, by placing the most discriminating splits near the root node, the tree does a better job of pruning during the query phase. So essentially, we need to search fewer nodes to ensure we have the exact nearest neighbor. The time complexity for building the tree is $O(n \lg n)$ when the median is computed by sorting at each individual level, ensuring the optimal splitting point.

In addition to the building phase, the k-d tree algorithm includes a query phase. In the query phase, we are trying to figure out, given our new point q and a target k of k nearest neighbors, what are the nearest neighbors of this new point? Essentially, how this works is that it maintains a running list of the k best candidates found so far, by sorting by the distance, essentially popping off any values that are irrelevant if they are further than all other k values. Starting at the root, the algorithm descends toward the leaf node that is most likely to contain a neighbor of q by comparing q 's coordinate along the current node's split axis to the node's split value. At each node, it checks whether or not the current node improves the candidate list, then always recurses into the nearby subtree, the side that q falls on. This is to ensure that we aren't missing a value that could just be close by, but on the other side of a partition. After visiting the near side, the algorithm then checks whether the splitting hyperplane is close enough to q that the far subtree might contain a better neighbor, essentially if the splitting hyperplane is further than the current furthest nearest neighbor. Then it would be impossible for there to be a point that exists beyond the hyperplane that could be closer than the current closest point. Thereby, we eliminate several values that need to be checked, and we only check values that are close to the point q , and can eliminate those that exist beyond the boundaries of certain hyperplanes. Or, in other words, we can prune large branches of the tree by recognizing that we don't need to access them to determine if they are closer than the current best options. The pseudocode for the query phase looks like this:

```
function kd_query(node, q, k, best_k):
    if node is null: return
    d = euclidean_distance(q, node.point)
    update best_k with (d, node.point) if better than current
    worst option
    diff = q[node.axis] - node.point[node.axis]
    near = node.left if diff < 0 else node.right
    far = node.right if diff < 0 else node.left
    kd_query(near, q, k, best_k)
    if len(best_k) < k or abs(diff) < best_k[-1][0]:
        kd_query(far, q, k, best_k)
```

3.2.2 Complexity.

In the best case, where we have low-dimensional data that is well distributed, the k-d tree query time is $O(\log n + k)$. The time complexity for building the tree is $O(n \log n)$ when the median is computed by sorting at each level. In the worst case, where we have high-dimensional data, the pruning conditions are unfortunately rarely satisfied, and the algorithm ends up degenerating towards $O(n)$, matching the worst-case speed of the brute force calculations. The empirical crossover point depends on the data and the

dimensionality. In this study, with 100,000 data points, it seems to occur around 20 dimensions with the k-d tree marginally exceeding brute force at $d=200$, which is relatively consistent with findings in the literature [1,2].

3.2.3 Implementation.

In this study, the k-d tree was implemented from scratch in Python using a custom `KDNode` class to store each node's point, split axis, and the left and right child references. The tree is built once per dimensionality test using the `build_kdtree()` function, and from that, the resulting structure was then used across 50 different queries for that dimensionality. Essentially, for 2 dimensions a tree was built, then for 5 dimensions, and so on up until 200 dimensions with datapoints. This was an important implementation decision, since rebuilding the tree on every query would have artificially inflated the runtime measurements. The build memory was measured separately from query memory using Python's `tracemalloc` module to capture peak heap allocation during each phase individually.

One notable implementation choice mentioned previously was the axis selection strategy. Rather than cycling through dimensions in a fixed order, the implementation instead always splits the dimension of the greatest spread among the current points. This ensures that the most discriminating splits are near the root and generally produces better pruning behavior in practice.

3.2.4 Strengths and Limitations.

The k-d tree's primary strength is in its ability to prune large portions of the search space in low-dimensional settings. This allows it to query significantly faster than the brute-force method when the number of dimensions, d , is small. The central limitation of this algorithm is that it is pretty sensitive to dimensionality increase. As we will see in the result, the k-d tree's query time actually ends up converging towards that of the brute force near $d=20$ and slightly exceeding it at $d=200$ due to the overhead of the tree traversal without any meaningful pruning. Hence, what is called the curse of dimensionality. For high-dimensional datasets, the k-d tree offers no practical advantage over the naive approach and introduces additional memory cost with no benefit to accuracy in the slightest.

4 APPROXIMATE NEAREST NEIGHBOR METHODS

Several approaches to approximate search have been proposed in the literature. Hajebi et al. [3] introduced a graph-based approach where a k-nearest neighbor graph is constructed offline and navigated using greedy hill-climbing

at query time, demonstrating competitive performance against k-d tree and LSH methods. More recently, Kazakovtsev et al. [5] proposed cluster-shaped indices for approximate search, showing that leveraging natural cluster structure in data can significantly improve both build and query efficiency. Malkov and Yashunin [6] proposed HNSW, a graph-based structure using hierarchical proximity graphs that achieves logarithmic search complexity in high-dimensional spaces.

4.1 Randomized K-D Tree Forest

The approximate nearest neighbor method implemented in this study is a randomized k-d tree forest. Essentially, here the core idea of this method is that, rather than just simply building a single k-d tree that always splits on the globally best axis, multiple trees are built where each tree's splits are determined by randomly sampling a subset of candidate axes at each node. In this instance, we are significantly increasing the number of hyperparameters in the methodology. The logic behind this method is that because each specific tree partitions the space differently, a query point that falls near a splitting hyperplane in one tree is likely to be handled correctly by another tree in the forest with a slightly different partition. At query time, all trees are searched, and their candidate sets are merged into one set, only returning the best k nearest neighbor results overall between all the trees. This approach is largely inspired by the randomized nearest neighbor search described by Lu and Gweon [4], who demonstrated that introducing randomness into axis selection reduces computational cost while maintaining high accuracy in high-dimensional settings. In addition, it also shares conceptual similarity to the ANNOY algorithm [9,10].

4.1.1 Algorithm.

The build phase is nearly identical to the standard k-d approach that we had previously. The main difference here is in how we are selecting the axis. Instead of always choosing the best axis, the algorithm randomly samples a fixed number of candidate axes and then picks from the best among those axes. Additionally, we have a second function that is introduced in the build phase, where we build the forest of trees. This function is really not that complicated; it essentially just calls the randomized `kdtree` multiple times so that we can create multiple trees.

```
function build_randomized_kdtree(points, depth=0,
candidate_axes=2):
    if points is empty: return null
    sampled = random_sample(range(ndim),
candidate_axes)
    axis = axis in sampled with largest spread
    sort points by axis value
    median = points[len(points) // 2]
```

```

node.left =
build_randomized_kdtree(points[:median_idx],
candidate_axes)
node.right =
build_randomized_kdtree(points[median_idx+1:],
candidate_axes)
return node

function build_forest(points, n_trees, candidate_axes):
forest = []
for i in range(n_trees):
tree = build_randomized_kdtree(points,
candidate_axes)
forest.append(tree)
return forest

```

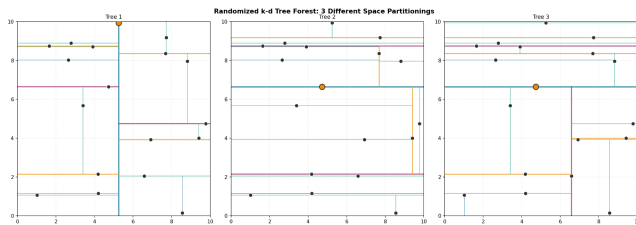


Figure 2: Three independent space partitionings produced by the randomized k-d tree forest on the same 2D dataset. Each panel represents one tree in the forest, with different partition structures resulting from randomized axis selection at each split. The diversity across trees ensures that a query point near a hyperplane boundary in one tree is likely to be handled correctly by another.

Notably, here we can set the number of candidate axes that are generated; with this hyperparameter, we can say that we want to choose between that many different axes as to which one we choose at each step. Under the normal kd-tree logic, this would just be set to be the maximum, or all of the axes are potential candidates. This, however, adds an element of randomness and should give us more of an approximation rather than a direct, accurate result for finding the k nearest neighbors. From there, we additionally have the query function, which is really effectively the same as the query function that we have in the k-d tree logic. In this case, however, it gathers all of the results for each tree independently. The results are pooled, deduplicated, sorted by distance, and only the k closest unique points are returned to the user.

```

function forest_query(forest, q, k):
candidates = []
for each tree in forest:
kd_query(tree, q, k, candidates)
deduplicate candidates by point identity
sort candidates by distance

```

return k closest candidates

Rather than using the standard backtracking query described in section 3.2.1, the forest implementation instead uses a depth-first search query module with a configurable backtracking budget. The core idea is that instead of always checking the far subtree whenever the splitting hyperplane is close enough, the algorithm instead limits the number of times that it's allowed to backtrack across splits. At $bt=0$, the query descends very greedily to the nearest leaf node and returns without any backtracking whatsoever. This makes the algorithm extraordinarily fast but also prone to missing true neighbors that may be just across the boundary partition. At $bt=1$ and $bt=2$, the algorithm is permitted to cross that boundary a limited number of times per path, so in essence it's able to recover some accuracy at a very substantial increase in runtime cost. The pseudocode for this algorithm is as follows:

```

function kd_query_dfs_backtrack(node, q, k, best_k,
backtracking):
if node is null: return
d = euclidean_distance(q, node.point)
update best_k with (d, node.point) if better than current
worst
diff = q[node.axis] - node.point[node.axis]
near, far = left/right based on diff
kd_query_dfs_backtrack(near, q, k, best_k,
backtracking)

if backtracking > 0 and (len(best_k) < k or abs(diff) <
best_k[-1][0]):
kd_query_dfs_backtrack(far, q, k, best_k,
backtracking - 1)

```

4.1.2 Complexity.

The computational complexity is like that of the k-d trees, where we need to separate the building of the trees and subsequently the forest from the query time. Once the trees are built, the actual query is all we actually need to do. In this case, the build complexity is $O(m \cdot n \log n)$, where $m=20$ trees. Query complexity per tree is $O(\log n + b)$, where b is the backtracking budget, giving a total query complexity of $O(m \cdot (\log n + b))$. Deduplication across trees adds $O(m \cdot k)$. In addition, there's also the overhead of merging and deduplicating the candidate lists to ensure we find the true k nearest neighbors. The memory usage scales pretty proportionally with the number of trees since each tree, of course, stores all n points in our case 100,000 independently in a binary tree structure.

4.1.3 Implementation.

Similar to the implementation that we had for the k-d tree, the forest and trees were implemented utilizing the same KNode class. Essentially, the main difference here is that we just need a modified build function that restricts the axis selection to a random sample of candidate dimensions, which is selected by a randomizer. For the purposes of this study, `n_trees` was set to 20, and the candidate axes were set to 2; this was used throughout the testing. Similar to previous methods, `tracemalloc` was used for calculating the amount of memory used for both the build phase and the query phase of the algorithm. The actual deduplication was handled by converting each candidate point to a tuple and then tracking the seen points in a set, where you can only have one of each specific value within a set. Then, resorting the candidate lists by distance and extracting the `k` nearest neighbors. Three backtracking budgets were tested: `bt=0`, `bt=1`, and `bt=2`, with `bt=5` and `bt=10` excluded as they produced runtimes exceeding 10 seconds at high dimensionalities due to near-exhaustive node exploration.

4.1.4 Strengths and Limitations.

The primary strength of this approach is its consistent query speed advantage over both exact methods across all tested dimensionalities. The backtracking budget provides a practical mechanism for trading accuracy against runtime, allowing the method to be tuned for the demands of a specific application. The central limitation here is that the accuracy degrades quite significantly with higher dimensionality datasets with low backtracking budgets. When looking at the no backtracking set for `d=200` dimensions, it falls to just a 1.2% accuracy. Higher backtracking budgets recover accuracy, but at an exponentially increasing runtime cost in higher dimensions, as the pruning condition to remove parts of the tree is almost never satisfied due to the high dimensionality of the dataset. Therefore, each backtrack can trigger pretty significant further recursive exploration in high-dimensional data. Backtracking is really only one of the hyperparameters in this algorithm, but it's quite a significant one at that. With good hyperparameter tuning, it's certainly possible to use this algorithm as an approximate nearest neighbor method for high-dimensional data with low runtime.

5 Experimental Methodology

5.1 Implementation

All five algorithms were implemented from scratch in Python without relying on any external nearest neighbor libraries. The runtime was measured by utilizing Python's `time.perf_counter()` for timing, and memory was tracked using the `tracemalloc` module, which records the peak heap

allocation during each phase. The build and query memory were viewed as separate modalities here, where the build memory was the cost of constructing the tree structure, and the query memory was the cost of executing a single search. All of which were averaged over the number of queries that were completed within the set. Accuracy was evaluated by comparing each algorithm's results against the brute-force ground truth, or in other words, whether the returned neighbors matched the brute-force result set exactly.

5.2 Dataset Generation

Rather than using uniformly random data, this study generates synthetic clustered datasets intended to resemble real-world data distributions. Cluster centers are placed uniformly at random in a hypercube of side length 10, and from there, individual points are generated by adding Gaussian noise with a standard deviation of 1.5 to a randomly selected cluster centroid. This ends up producing data that has a more natural local structure that is meant to challenge the algorithm more realistically than, for instance, uniformly distributed random data would. Since items belonging to one cluster could potentially be equidistant from neighbors belonging to other clusters. This potentially could be seen as another hyperparameter to tweak and may need further refinement as well, however, the idea is to try and create a dataset that's relatively realistic.

For each tested dimensionality, a dataset of 100,000 points was generated along with 50 query points drawn from the same clustered distribution. `k=10` was used throughout, meaning each query searches for its 10 nearest neighbors in the dataset.

5.3 Accuracy Measurement

Since brute-force search is exact by definition, the results of the brute-force search can serve as a ground truth for accuracy evaluation. For each query, accuracy is specifically defined as the fraction of the algorithm's returned `k` neighbors or what the algorithm believes are the `k` nearest neighbors that also appear in the set of numbers within the brute force result set.

$$\text{accuracy} = |\text{result} \cap \text{ground_truth}| / k$$

Therefore, if all of the values in our results return as true in this case, then the accuracy will be 1. This is computed per query and then averaged across all 50 different queries for each dimensionality in order to get the most accurate results.

5.4 Tested Dimensionality

For this project, seven different dimensionalities were tested, each with 100,000 points: 2, 5, 10, 20, 50, 100, and 200. These were chosen in order to try and span the range from low-dimensional settings where tree-based methods

are known to excel, into the intermediate range where the degradation of certain models really begins, all the way into the higher-dimensional regime where we can look at the curse of dimensionality in its full effect.

6 Results

6.1 Query Runtime

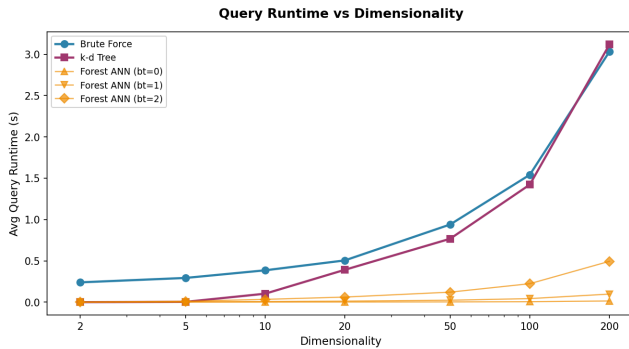


Figure 3: Average query runtime (seconds) as a function of dimensionality for all five algorithms.

Dim	BF	kdTree	ANN 0	ANN 1	ANN 2
2	0.2397	0.0001	0.0005	0.001	0.0013
5	0.293	0.0035	0.0007	0.0038	0.0107
10	0.3854	0.1023	0.001	0.0073	0.0338
20	0.5052	0.3915	0.0016	0.0124	0.0616
50	0.9397	0.7671	0.003	0.0233	0.1211
100	1.5414	1.4225	0.0056	0.0431	0.2234
200	3.0325	3.1177	0.0144	0.0978	0.4945

Table 1: Average Query Runtime (Seconds)

At $d=2$, the k-d Tree is dramatically faster than the brute force variant, completing queries thousands of times faster than its brute force counterpart. All three forest ANN variants are also faster than the brute force as well, with the highest being $bt=2$, only 13x slower than utilizing a k-d Tree. As the dimensionality increases, however, the k-d Tree really rapidly degrades, even surpassing the runtime of the brute force strategy at $d=200$ with a runtime of 3.03s vs 3.12s, showcasing the curse of dimensionality in action. All three of the ANN methods remain consistently significantly faster than either of the exact methods across every single tested dimensionality. At $d=200$, $bt=0$ completes queries in

0.01 seconds, albeit with very low accuracy. The other two, $bt=1$ and $bt=2$, are slower than the first but still substantially faster than either of the exact methods, at 0.10s and 0.49s, respectively. A 6x speedup over the brute force approach at $d=200$.

6.2 Memory Usage

Dim	BF	kd Tree	ANN
2	0	12505.26	205476.6
5	0	12500.64	205476.6
10	0	12500.64	205476.6
20	0	12500.64	205476.6
50	0	12500.64	205470.1
100	0	12500.64	205469.9
200	0	12500.64	205470.7

Table 2: Build Memory (KB)

Dim	BF	kdTree	ANN 0	ANN 1	ANN 2
2	10045	1.09	6.05	6.27	6.28
5	10045	1.1	15.19	7.7	7.2
10	10045	1.1	15.19	14.34	7.53
20	10045	1.1	15.19	15.43	14.65
50	10045	1.1	99.32	95.42	74.89
100	10045	1.1	176.17	173.75	151.34
200	10045	1.1	333.68	327.54	299.42

Table 3: Query Memory (KB)

Brute force is the only algorithm that requires no build memory but consistently uses approximately 10,045 KB per query to store and sort the full distance list across all dimensionalities. The k-d Tree, on the other hand, requires around 12,500 KB of build memory but only a very small fraction of memory on every query, ~1.1 KB, reflecting the efficiency of the tree traversal itself as well as the efficiency in setting up a data structure prior to searching. The forest approach requires approximately 205,000 KB of build memory, which is roughly 15x the size of the k-d tree, which makes sense given that it is just 20 k-d trees stored with

100,000 points each. Query memory for the forest increases with dimensionality from 6KB at $d=2$ all the way up to 334KB at $d=200$, which is actually reflective of the growing size of the candidate point tuples as the dimensionality increases. Given that the algorithm finds k nearest neighbors 20 times and takes the best of that set. This kind of growth, however, does not occur in the k -d tree because it maintains a fixed list of exactly $k=10$ candidates during the traversal itself, popping off unnecessary candidates as it explores.

6.3 Accuracy

Dim	BF	kdTree	ANN 0	ANN 1	ANN 2
2	1	1	0.372	0.826	0.996
5	1	1	0.676	0.974	0.998
10	1	1	0.412	0.898	0.99
20	1	1	0.124	0.616	0.942
50	1	1	0.048	0.31	0.76
100	1	1	0.02	0.184	0.558
200	1	1	0.012	0.106	0.392

Table 4: Accuracy

Unlike the exact methods, which maintain perfect accuracy across all dimensionalities by definition, the three ANN variants exhibit a clear and meaningful accuracy-speed tradeoff, which is controlled by the backtracking budget hyperparameter. At low dimensionalities, $bt=2$ achieves a nearly perfect accuracy of 99.6% and 99.8% accuracy at $d=5$ while still remaining drastically faster than the brute force algorithm. Accuracy degrades, however, with dimensionality for all three variants, most severely for $bt=0$, which falls to really only being 1.2% at $d=200$. However, even in the high dimensionality datasets, $bt=2$ retains close to 40% accuracy, representing a pretty substantial albeit incomplete recovery of the dataset.

7 Discussion

7.1 K-D Tree Degradation

The k -d tree results closely match theoretical predictions from the literature. At $d=2$ and $d=5$, the tree's pruning is highly effective; most of the dataset can be eliminated simply by comparing the query's distance to splitting hyperplanes, and the algorithm traverses only a small fraction of the total nodes. This produces a dramatic

speedup over the brute force method at low dimensionalities. However, as dimensionality increases toward $d=10$, the geometry of the space works against the pruning logic. In higher-dimensional spaces, the distance from a query point to any given splitting hyperplane tends to be small relative to the distance to the current best nearest neighbor candidate. This means that the condition required to safely prune the farther subtree is almost never actually satisfied in reality. Therefore, the algorithm is forced to explore both subtrees at nearly every node, so it goes from being an effective, efficient search with pruning to essentially being a brute force search with the additional overhead of the recursive function calls and tree traversal. By the time you reach 200 dimensions, this overhead actually makes the k -d tree marginally slower than brute force search, at 3.12s vs 3.03s respectively, which is a pretty meaningful result. It means that beyond a certain number of dimensions its actually a far worse implementation than just implementing the naive approach. This degradation pattern is consistent with the analysis presented by Sample et al. [2], who showed that k -d tree search encounters exponentially more nodes as dimensionality increases, even under optimized search strategies.

7.2 Forest ANN

The most significant results of this study come from the three forest ANN variants and the role of the backtracking budget as a hyperparameter. Unlike the previous iteration of this study, which used a standard backtracking query and achieved perfect accuracy at the cost of actually being the slowest method, the DFS module with a configurable backtracking hyperparameter produces an actually meaningful accuracy-to-speed tradeoff.

At $bt=0$, the algorithm is extraordinarily fast, completing queries 200x faster than the brute force method for 200 dimensions. The accuracy also collapses entirely with dimensionality, falling to just 1.2% at 200 dimensions. This is because, without any backtracking, the algorithm commits entirely to whichever partition the query point falls into at each split, and in high dimensions, the true nearest neighbors are increasingly likely to lie just across a partition boundary that the algorithm never crosses at all. The other two, $bt=1$ and $bt=2$, actually do recover meaningful accuracy at a proportional runtime cost. At $d=5$, $bt=1$ achieves 97.4% accuracy, and $bt=2$ achieves 99.8% accuracy while both remain dramatically faster than the brute force method. The degradation with dimensionality, however, is unavoidable, but the backtracking budget gives the user direct control over where the accuracy vs speed curve they want to operate.

7.3 Implications for Algorithm Selection

These results support a practical framework for nearest neighbor algorithm selection. For small datasets or applications where simplicity is paramount, brute force is

entirely reasonable and would always yield a correct result; however, even then, it still might be better to utilize a k-d tree because for low to medium-dimensional datasets that have even a large amount of values, the k-d tree offers a meaningful speedup and additionally would also yield exact results. In very high dimensionality settings where you are dealing with large-scale datasets, such as in modern machine learning applications [7] in big data, neither of the first two methods would really scale acceptably, and this is where the usefulness of ANN methods really shines. This is particularly relevant in the context of modern applications such as those described by Kazakovtsev et al. [5], where approximate methods with cluster-aware indexing have demonstrated practical advantages at scale. The backtracking budget in the forest ANN adds a further dimension to this framework. For applications where some accuracy loss is acceptable, $bt=0$ or $bt=1$ offer dramatic speedups over any exact method at scale. For applications requiring higher accuracy, $bt=2$ provides a middle ground, achieving near-perfect accuracy at low dimensions and meaningful accuracy retention even at $d=200$, while simultaneously still outperforming both of the exact methods in runtime.

7.4 Limitations of This Study

Several limitations of this study are worth acknowledging. 100,000 points is a more realistic dataset size than the 5,000 used previously during testing. It still falls short of the scale at which ANN methods are most commonly deployed in practice, such as million-scaled embedding databases. The backtracking hyperparameter was only tested at $bt=0, 1$, and 2 due to the exponential runtime cost at higher values, such as 5 and 10, where it would actually slow down more than even a brute force approach. A more systematic exploration of fractional or adaptive backtracking strategies could yield better accuracy to speed tradeoffs. Additionally, the clustered data distribution may still make partitioning of the data easier than a truly adversarial or uniformly distributed dataset would. Finally, only Euclidean distance was evaluated in this study; other metrics, such as the Manhattan distance, may interact differently with the DFS query module at high dimensionalities.

8 CONCLUSION AND FUTURE WORK

8.1 Conclusion

This study implemented and experimentally evaluated five k-nearest neighbor search algorithms, brute-force search, k-d tree search, and three configurations of a randomized k-d tree forest with a novel DFS query module with configurable backtracking budgets of $bt=0$, $bt=1$, and $bt=2$, across a synthetic clustered dataset of 100,000 points spanning multiple dimensions from $d=2$ to $d=200$. The k-d tree demonstrated clear advantages over brute force at low dimensionalities, with a significant speedup at $d=2$ through effective spatial pruning. However, this advantage eroded

quite rapidly with increasing dimensionality, converging towards brute force search performance starting at $d=20$ and surpassing it by $d=200$. All three forest ANN variants were consistently faster than both of the exact methods across every dimensionality, with $bt=0$ achieving up to a 200x speedup over the brute-force strategy in 200 dimensions, albeit at the cost of near-zero accuracy. The backtracking hyperparameter implementation emerged as a meaningful and practical hyperparameter: $bt=2$ retained a 39.2% accuracy at $d=200$ while remaining roughly 6x faster than the brute force strategy, and achieved near-perfect accuracy at low dimensionalities. These results demonstrate that the DFS query module with configurable backtracking as a hyperparameter provides a genuine and tunable accuracy to speed tradeoff that was absent in the previous iteration of this study.

8.2 Future Work

Several directions for future work emerge quite naturally from these results. Future experiments should evaluate these methods on datasets ranging from $n=1,000,000$ and beyond to further validate the scalability advantages of the ANN approach. This testing should really elucidate the effectiveness of utilizing the ANN algorithm, while simultaneously it should additionally showcase how impractical these other methods are for large high-dimensional datasets. A more systematic exploration of the backtracking budget hyperparameter, including adaptive or fractional backtracking strategies, to better characterize the accuracy-speed tradeoff across dimensionalities, should be pursued. There may be a bug currently in the implementation where it applies DFS to previously visited nodes; as such, adding a tracker or creating a queue of nodes to visit may actually increase the strength of the method even further. Additionally, it would be very beneficial to do a more systematic exploration of the forest hyperparameters, specifically the number of trees `n_trees` and candidate `axes`. It would be interesting to clarify the accuracy vs. efficiency tradeoff more precisely and potentially reveal configurations where the forest offers a more genuine speed advantage. Next, implementing and comparing more sophisticated ANN methods such as ANNOY[9,10], Locality Sensitive Hashing, or graph-based methods like HNSW[6] would provide a broader context for where the randomized forest approach actually sits within the landscape of approximate search. Finally, evaluating these algorithms on real-world high-dimensionality datasets such as word embeddings, image feature vectors, or medical databases would help to validate whether the patterns observed here on synthetic data generalize to actual practical use cases.

REFERENCES

- [1] J. L. Bentley. 1975. Multidimensional binary search trees used for associative searching. *Communications of the ACM* 18, 9 (1975), 509–517.
- [2] N. Sample, M. Haines, M. Arnold, and T. Purcell. 2001. Optimizing search strategies in k-d trees. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 23, 6 (2001), 631–645.

- [3] K. Hajebi, Y. Abbasi-Yadkori, H. Shahbazi, and H. Zhang. 2011. Fast approximate nearest-neighbor search with k-nearest neighbor graph. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 1312–1319.
- [4] J. Lu and H. Gweon. 2019. Random k conditional nearest neighbor for high-dimensional data. *Pattern Recognition* 96 (2019).
- [5] V. Kazakovtsev et al. 2025. Fast adaptive approximate nearest neighbor search with cluster-shaped indices. *Big Data and Cognitive Computing* 9, 10 (2025).
- [6] Y. A. Malkov and D. A. Yashunin. 2018. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2018), 824–836.
- [7] I. Goodfellow, Y. Bengio, and A. Courville. 2016. *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>
- [8] R. Suwanda, Z. Syahputra, and E. M. Zamzami. 2020. Analysis of Euclidean Distance and Manhattan Distance in the K-Means Algorithm for Variations Number of Centroid K. *Journal of Physics: Conference Series* 1566 (2020), 012058.
- [9] M. Aumüller, E. Bernhardsson, and A. Faithfull. 2017. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. In *Similarity Search and Applications (SISAP 2017)*, Lecture Notes in Computer Science, vol. 10609. Springer, Cham. https://doi.org/10.1007/978-3-319-68474-1_3
- [10] E. Bernhardsson. 2015. Annoy: Approximate Nearest Neighbors in C++/Python. GitHub. <https://github.com/spotify/annoy>
- [11] Python Software Foundation. 2024. Sorting HOW TO. Python Documentation. <https://docs.python.org/3/howto/sorting.html>

AI Usage Disclosure:

AI tools were used in a limited capacity, as outlined in the syllabus usage policy, throughout the creation of this project. Grammarly was used for grammar and spelling review throughout the writing process. ChatGPT was utilized for help with brainstorming and creating an outline for the paper. Additionally, I utilized ChatGPT and Grammarly in some sections to help my writing sound clearer and more cohesive. Gemini was in part utilized for finding sources, but all sources for this paper were verified and manually reviewed directly. The ideas, interpretations, and conclusions represented in this paper are my own. Additionally, I did utilize Claude in the coding; this is outlined in the code itself, but I will reiterate it here. It was used for helping to set up visualizations, creating tests to use, and, additionally, for debugging at times. All of the algorithms themselves, however, and their implementations were designed by me.